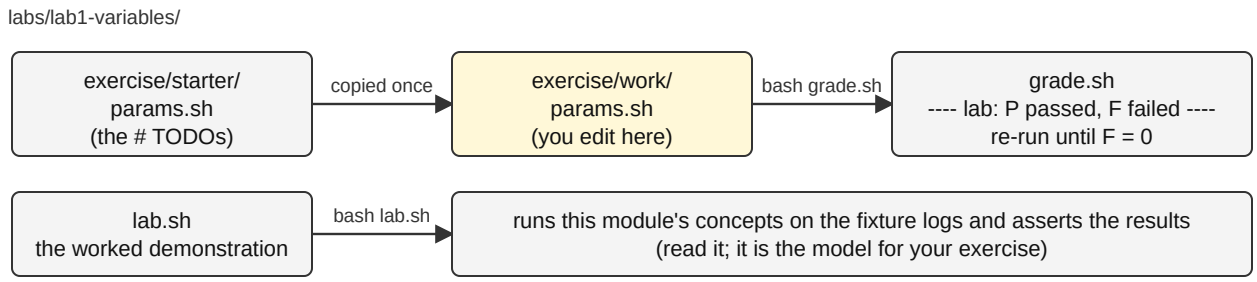
Lab 1 — Variables, Quoting and Parameter Expansion: Three Helpers (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

All paths are relative to `labs/lab1-variables/exercise/`.

File	What it is	You edit it?
`TASK.md`	The short statement of the exercise	No
`starter/params.sh`	Three functions that return `FIXME`, each with a `# TODO`	No — the grader copies it
`work/params.sh`	**Your** copy (created by the first `grade.sh` run)	**Yes**
`grade.sh`	The grader: 7 checks. It *sources* your file and calls the functions	No
`work/REFLECTION.md`	Your two reflection answers	**Yes** (you create it)

Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Build `stem` — turn a log path into a clean name with parameter expansion.
- **Part 3:** Build `report_name` — compose a file name from `stem` and a date.
- **Part 4:** Build `parse_kv` — read one `key = value` line into two variables, trimmed.
- **Part 5:** Verify with the grader; write the two reflection answers.

Background / Scenario

`logsleuth` names its report files after the log it analysed (`report-access-2026-10-16.txt`) and reads its settings from a small `key = value` configuration file. Both jobs are done with **parameter expansion** — bash's built-in way of trimming and reshaping a value without starting another program. In this lab you write the three helpers `logsleuth` really uses: `stem`, `report_name` and `parse_kv`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

The grader *sources* `params.sh` and calls the functions directly, so the file only has to **define** the functions. The functions must print their result (or, for `parse_kv`, set the two global variables `KEY` and `VALUE`) exactly as the checks expect.

Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready`.
- Module M01, steps S01.4–S01.6.
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

> **Instructor Note:** Budget 45 minutes; grading ≈ 5 minutes per learner. Observed: the starter scores `---- lab1: 0 passed, 7 failed ----` (every FAIL shows `got [FIXME]`); the reference scores 7/7. The trap to watch for: a `parse_kv` that declares `KEY`/`VALUE` with `local` **fails** — the grader reads the globals — which is a good preview of module M04's scope lesson.

Part 1: Prepare the exercise folder

Step 1: Run the grader once.

```
cd labs/lab1-variables/exercise
bash grade.sh
```

Expected: `---- lab1: 0 passed, 7 failed ----`, every FAIL showing `expected [...], got [FIXME]`. The checks run top to bottom: three for `stem`, one for `report_name`, three for `parse_kv`. Fix one function at a time.

Step 2: Open `work/params.sh`.

Each function has a `# TODO` comment quoting the move it needs. The file must not run anything at the top level — it is sourced.

Part 2: Build `stem`

`stem PATH` turns a log path into a clean name: `/var/log/nginx/access.log` → `access`; `auth.log` → `auth`; `a/b/c/app` → `app` (nothing to drop).

Step 1: Strip the directory, then the suffix.

Replace the body of `stem` with:

```
stem() {
  local path="$1" base
  base="${path##*/}"              # strip everything up to the last '/'
  printf '%s\n' "${base%.log}"    # strip a trailing '.log'
}
```

How it works: `${path##*/}` removes the **longest** match of `*/` from the front — the whole directory part — leaving the file name. `${base%.log}` removes the **shortest** match of `.log` from the end; a name without that suffix is left untouched.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Note: One `#` would remove the *shortest* leading match (`/`), leaving `var/log/nginx/access.log`. The doubled `##` is what makes it the basename.

Step 2: Try it by hand.

```
source work/params.sh
stem /var/log/nginx/access.log; stem auth.log; stem a/b/c/app
```

Expected: `access`, `auth`, `app`.

Part 3: Build `report_name`

`report_name PATH DATE` prints `report-<stem>-<date>.txt`.

Step 1: Compose the name.

```
report_name() {
  local path="$1" date="$2"
  printf 'report-%s-%s.txt\n' "$(stem "$path")" "$date"
}
```

`$(stem "$path")` captures what `stem` prints and drops it into the format. Each value is a separate, quoted `printf` argument.

Step 2: Try it.

```
report_name /var/log/nginx/access.log 2026-10-16
```

Expected: `report-access-2026-10-16.txt`.

Part 4: Build `parse_kv`

`parse_kv "top_n = 25"` must set the global variables `KEY=top_n` and `VALUE=25` with surrounding whitespace removed — also for a line like `" brute_threshold=10 "`.

Step 1: Split on `=`, then trim.

```
parse_kv() {
  local line="$1" k v
  IFS='=' read -r k v <<< "$line"
  read -r KEY   <<< "$k"          # a bare read trims surrounding whitespace
  read -r VALUE <<< "$v"
}
```

How it works: `IFS='=' read -r k v <<< "$line"` splits the line at the first `=` into `k` and `v` (the `IFS='='` applies only to that one command). A second, plain `read -r` into each global strips leading and trailing spaces and tabs for free, because `read` uses the default `IFS` to trim.

Note: `KEY` and `VALUE` are deliberately **not** declared `local` — the caller needs them after the function returns. A `local` would make the grader see nothing.

Step 2: Try it.

```
parse_kv "top_n = 25"; echo "[$KEY] [$VALUE]"
parse_kv "  brute_threshold=10  "; echo "[$KEY] [$VALUE]"
```

Expected: `[top_n] [25]` and `[brute_threshold] [10]` — no spaces inside the brackets.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Part 5: Verify and reflect

Step 1: Run the grader until it is green.

```
bash grade.sh
```

Expected: `---- lab1: 7 passed, 0 failed ----`. The seven checks: `stem` on `/var/log/nginx/access.log`, `auth.log` and `a/b/c/app`; `report_name → report-access-2026-10-16.txt`; `parse_kv` key; `parse_kv` value; `parse_kv` trims around = and the whole line.

Symptom	Cause	Fix
`stem` returns `var/…`	A single `#` strips the shortest match	Use `##`
`stem a/b/c/app` returns something other than `app`	The suffix removal is not `%.log`	`${base%.log}` leaves a name without `.log` untouched
`parse_kv` key or value keeps spaces	The second `read` is missing	Print `[$k]` and `[$v]` to see the spaces, then add the trimming reads
`parse_kv` checks see empty values	`KEY`/`VALUE` were declared `local`	Remove the `local`

Step 2: Reflection.

Create `work/REFLECTION.md` and answer in your own words (3–5 sentences each):

Question 1: Why do `${1:-world}` and `${x:-10}` use a colon? What changes without it?

> **Answer:** **Met:** the colon makes the default apply when the variable is *empty as well as unset*; without it an empty value is kept. **Strong:** gives the observed pair (`${x-10}` → empty line, `${x:-10}` → `10`) or names a case where the difference matters (an optional argument passed as `""`). **Not met:** "the colon is required syntax" or a restatement of the symbol.

Question 2: In S01.2 an unquoted `$path` became two arguments. What decides *where* an unquoted value splits, and why is quoting — not changing `IFS` — the right fix?

> **Answer:** **Met:** an unquoted expansion is split on the characters in `IFS` (and then globbed); quoting hands the value over as one argument regardless of IFS. **Strong:** notes that changing IFS only changes *which* characters split (tabs and newlines still do, and `*` still globs), so it is not a fix. **Not met:** "quotes make it a string" or "IFS should be set to avoid splitting" (the misconception to correct).
>
> *Calibration.* Met: "Bash splits an unquoted `$path` wherever it finds a character from IFS — a space by default — and then expands wildcards; `"$path"` stops both, so the value stays one argument." Not met: "You should always quote variables because it is best practice." — a rule without the mechanism; return it with "what *decides* where the split happens?".

Hand-in and grading

Hand in `work/params.sh`, `work/REFLECTION.md` and your final grader output. **Pass** = `0 failed` **and** both reflection answers name the mechanism. Unlimited attempts.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Grading procedure (≈5 min per learner).

1. `GRADE_TARGET=/path/to/their/work bash labs/lab1-variables/exercise/grade.sh` → require `---- lab1: 7 passed, 0 failed ----`.
2. Read `REFLECTION.md` against the descriptors. Record `A-02 PASS` only with 7/7 and both answers at Met or better.
3. **Acceptable variations:** `stem` using `basename`/`sed` passes the checks but misses the point — coach toward expansion. A `parse_kv` that trims with `${k#"${k%%[![:space:]]*}"}` idioms is correct but over-complex at this level. A `parse_kv` that `local`-izes `KEY`/`VALUE` **fails** — the grader reads globals.
4. **Model feedback (Not yet):** "`stem a/b/c/app` returned `var/`…? Your `#` strips the shortest match; you need `##`." Remediation: S01.4 and S01.6.
5. **Equivalent reassessment:** same grader; Q1′ "Why does `parse_kv` set `KEY`/`VALUE` as globals rather than printing them?" (Met: two values to return; stdout would need parsing; M04 discusses the trade-off) and Q2′ "Give one input that is safe under `IFS=$'\n\t'` but still breaks unquoted." (Met: a value containing `*`, or a tab).

Instructor Note: Appendix — reference solution (`solution/params.sh`, observed 7/7):

```
#!/usr/bin/env bash
# Lab 1 solution — parameter-expansion helpers, the exact moves logsleuth uses.

stem() {
  local path="$1" base
  base="${path##*/}"              # strip everything up to the last '/'
  printf '%s\n' "${base%.log}"    # strip a trailing '.log'
}

report_name() {
  local path="$1" date="$2"
  printf 'report-%s-%s.txt\n' "$(stem "$path")" "$date"
}

parse_kv() {
  local line="$1" k v
  IFS='=' read -r k v <<< "$line"
  read -r KEY   <<< "$k"          # a bare read trims surrounding whitespace
  read -r VALUE <<< "$v"
}
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.